

EXPERIMENT-5

Dynamic Programming using Policy Iteration and Value Iteration

Name: AKHILA POCHIMI REDDY

Reg no: 192425225

Course code: MLA0305

Course Name: REINFORCEMENT LEARNING

Slot: B

Aim

To implement **Policy Iteration and Value Iteration** using Dynamic Programming and compare their performance in finding the optimal path from **START to GOAL**.

Procedure

1. Create a Grid World with defined **START and GOAL** states.
2. Define the available actions, rewards, discount factor, and state transitions.
3. Implement **Value Iteration** to calculate the optimal value function and policy.
4. Implement **Policy Iteration** through policy evaluation and policy improvement.
5. Find the path from START to GOAL using both methods.
6. Compare the methods using a **summary table and comparison graph**.

Python code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from matplotlib.patches import Rectangle

from IPython.display import display

# =====

# EXPERIMENT 5

# Dynamic Programming using Policy Iteration and Value Iteration
```

```
# =====

N = 4

states = N * N

actions = 4

gamma = 0.9

theta = 0.001

START = 0

GOAL = 15

# Actions: Up, Down, Left, Right

moves = {

    0: (-1, 0),

    1: (1, 0),

    2: (0, -1),

    3: (0, 1)

}

arrows = {

    0: "↑",

    1: "↓",

    2: "←",

    3: "→"

}

# =====

# STATE FUNCTIONS

# =====
```

```

def get_position(s):

    return divmod(s, N)

def get_state(row, col):

    return row * N + col

# =====

# TRANSITION FUNCTION

# =====

def next_state(s, action):

    if s == GOAL:

        return GOAL

    row, col = get_position(s)

    dr, dc = moves[action]

    new_row = max(0, min(N - 1, row + dr))

    new_col = max(0, min(N - 1, col + dc))

    return get_state(new_row, new_col)

# =====

# REWARD FUNCTION

# =====

def get_reward(next_s):

    if next_s == GOAL:

        return 0

    return -1

# =====

# VALUE ITERATION

```

```

# =====

V_vi = np.zeros(states)

vi_changes = []

while True:

    V_new = V_vi.copy()

    for s in range(states):

        if s == GOAL:

            continue

        action_values = []

        for a in range(actions):

            ns = next_state(s, a)

            value = (

                get_reward(ns)

                + gamma * V_vi[ns]

            )

            action_values.append(value)

        V_new[s] = max(action_values)

    change = np.max(np.abs(V_new - V_vi))

    vi_changes.append(change)

    V_vi = V_new

    if change < theta:

        break

# Find Value Iteration policy

policy_vi = np.zeros(states, dtype=int)

```

```

for s in range(states):
    if s == GOAL:
        continue

    action_values = []

    for a in range(actions):
        ns = next_state(s, a)

        value = (
            get_reward(ns)
            + gamma * V_vi[ns]
        )

        action_values.append(value)

    policy_vi[s] = np.argmax(action_values)

# =====

# POLICY ITERATION

# =====

policy_pi = np.zeros(states, dtype=int)
V_pi = np.zeros(states)
pi_changes = []
policy_iterations = 0

while True:

    # -----

    # POLICY EVALUATION

    # -----

    while True:

```

```

V_new = V_pi.copy()

for s in range(states):

    if s == GOAL:

        continue

    a = policy_pi[s]

    ns = next_state(s, a)

    V_new[s] = (

        get_reward(ns)

        + gamma * V_pi[ns]

    )

    change = np.max(

        np.abs(V_new - V_pi)

    )

    pi_changes.append(change)

V_pi = V_new

if change < theta:

    break

# -----

# POLICY IMPROVEMENT

# -----

stable = True

for s in range(states):

    if s == GOAL:

        continue

```

```

old_action = policy_pi[s]

action_values = []

for a in range(actions):
    ns = next_state(s, a)

    value = (
        get_reward(ns)
        + gamma * V_pi[ns]
    )

    action_values.append(value)

policy_pi[s] = np.argmax(action_values)

if old_action != policy_pi[s]:
    stable = False

policy_iterations += 1

if stable:
    break

# =====

# FIND PATH

# =====

def find_path(policy):
    path = [START]

    current = START

    for _ in range(30):
        if current == GOAL:
            break

```

```

        current = next_state(
            current,
            policy[current]
        )

        path.append(current)

    return path

path_pi = find_path(policy_pi)
path_vi = find_path(policy_vi)

# =====

# SUMMARY TABLE

# =====

summary = pd.DataFrame({
    "Method": [
        "Policy Iteration",
        "Value Iteration"
    ],
    "Iterations": [
        policy_iterations,
        len(vi_changes)
    ],
    "Path Length": [
        len(path_pi) - 1,
        len(path_vi) - 1
    ],

```



```

    "Start Value": [
        round(V_pi[START], 3),
        round(V_vi[START], 3)
    ]
})

print("\n" + "=" * 55)

print("        SUMMARY")

print("=" * 55)

display(summary)

# =====

# STATE VALUE COMPARISON TABLE

# =====

state_summary = pd.DataFrame({
    "State": [
        f"S{i+1}" for i in range(states)
    ],
    "Policy Iteration": np.round(
        V_pi, 2
    ),
    "Value Iteration": np.round(
        V_vi, 2
    )
})

print("\n" + "=" * 55)

```

```

print("    STATE VALUE COMPARISON")

print("=" * 55)

display(state_summary)

# =====

# GRID WORLD VISUALIZATION

# =====

fig, axes = plt.subplots(
    1, 2,
    figsize=(12, 6)
)

def draw_grid(ax, policy, path, title):
    ax.set_xlim(-0.5, N - 0.5)
    ax.set_ylim(N - 0.5, -0.5)
    ax.set_aspect("equal")
    # Draw grid
    for row in range(N):
        for col in range(N):
            rectangle = Rectangle(
                (col - 0.5, row - 0.5),
                1,
                1,
                fill=False,
                linewidth=2
            )

```

```
ax.add_patch(rectangle)
```

```
s = get_state(row, col)
```

```
# START
```

```
if s == START:
```

```
    ax.text(
```

```
        col,
```

```
        row - 0.22,
```

```
        "START",
```

```
        ha="center",
```

```
        fontsize=9,
```

```
        fontweight="bold"
```

```
    )
```

```
    ax.text(
```

```
        col,
```

```
        row + 0.18,
```

```
        "●",
```

```
        ha="center",
```

```
        fontsize=18
```

```
    )
```

```
# GOAL
```

```
elif s == GOAL:
```

```
    ax.text(
```

```
        col,
```

```
        row,
```

```

        "GOAL",

        ha="center",

        va="center",

        fontsize=10,

        fontweight="bold"

    )

# Policy arrows

else:

    ax.text(

        col,

        row,

        arrows[policy[s]],

        ha="center",

        va="center",

        fontsize=23,

        fontweight="bold"

    )

# Draw actual path

path_x = [

    get_position(s)[1]

    for s in path

]

path_y = [

    get_position(s)[0]

```

```
        for s in path
    ]
    ax.plot(
        path_x,
        path_y,
        linestyle="--",
        linewidth=2,
        marker="o",
        markersize=5
    )
    ax.set_title(
        title,
        fontsize=14,
        fontweight="bold"
    )
    ax.set_xticks([])
    ax.set_yticks([])
    # Policy Iteration
    draw_grid(
        axes[0],
        policy_pi,
        path_pi,
        "Policy Iteration"
    )
```

```

# Value Iteration

draw_grid(
    axes[1],
    policy_vi,
    path_vi,
    "Value Iteration"
)

fig.suptitle(
    "START → GOAL: Policy Iteration vs Value Iteration",
    fontsize=16,
    fontweight="bold"
)

plt.tight_layout()

plt.show()

# =====

# COMPARISON GRAPH

# =====

methods = [
    "Policy Iteration",
    "Value Iteration"
]

path_lengths = [
    len(path_pi) - 1,
    len(path_vi) - 1

```

```

]

iteration_counts = [

    policy_iterations,

    len(vi_changes)

]

x = np.arange(len(methods))

width = 0.35

fig, ax = plt.subplots(

    figsize=(9, 5)

)

bars1 = ax.bar(

    x - width / 2,

    path_lengths,

    width,

    label="Path Length"

)

bars2 = ax.bar(

    x + width / 2,

    iteration_counts,

    width,

    label="Iterations"

)

ax.set_xlabel(

    "Dynamic Programming Method"

```

```

)

ax.set_ylabel(

    "Count"

)

ax.set_title(

    "Policy Iteration vs Value Iteration",

    fontsize=14,

    fontweight="bold"

)

ax.set_xticks(x)

ax.set_xticklabels(methods)

ax.legend()

ax.grid(

    axis="y",

    linestyle="--",

    alpha=0.4)

# Values above bars

for bars in [bars1, bars2]:

    for bar in bars:

        height = bar.get_height()

        ax.text(

            bar.get_x()

            + bar.get_width() / 2,

            height + 0.05,

```



```

        f'{height:.0f}',
        ha="center",
        fontweight="bold"
    )
plt.tight_layout()
plt.show()

# =====

# RESULT

# =====

print("\n" + "=" * 55)
print("          RESULT")
print("=" * 55)
print(
    "Policy Iteration and Value Iteration were successfully "
    "implemented. Both methods found the path from START "
    "to GOAL, and their path length and iteration performance "
    "were compared using a summary table and comparison graph."
)

```

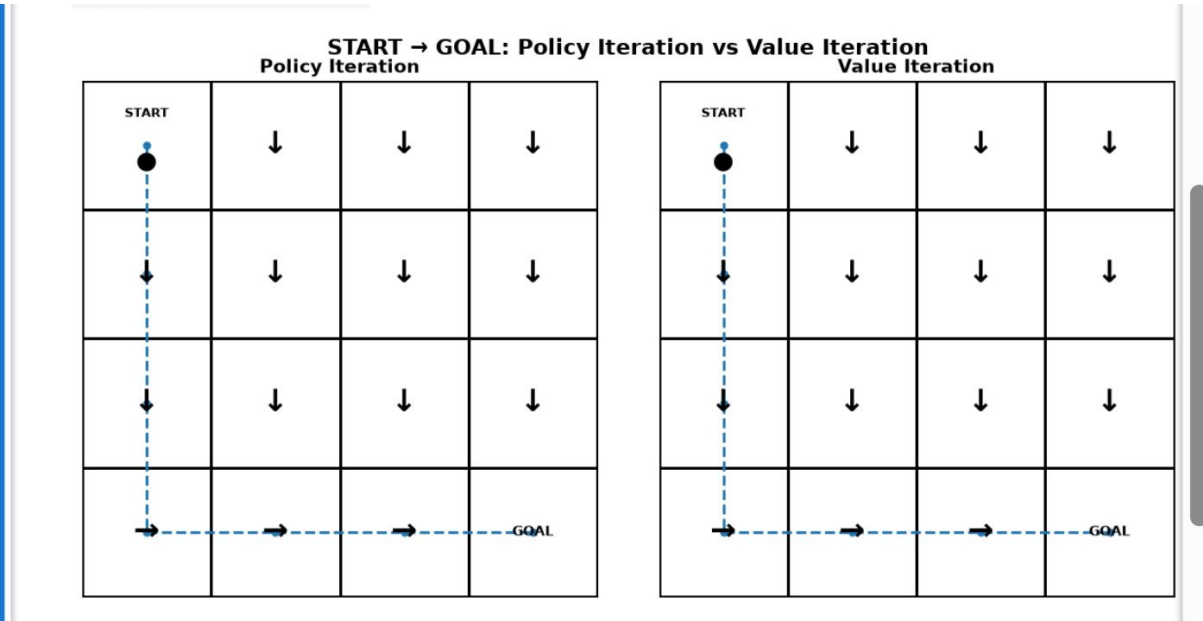
Output:

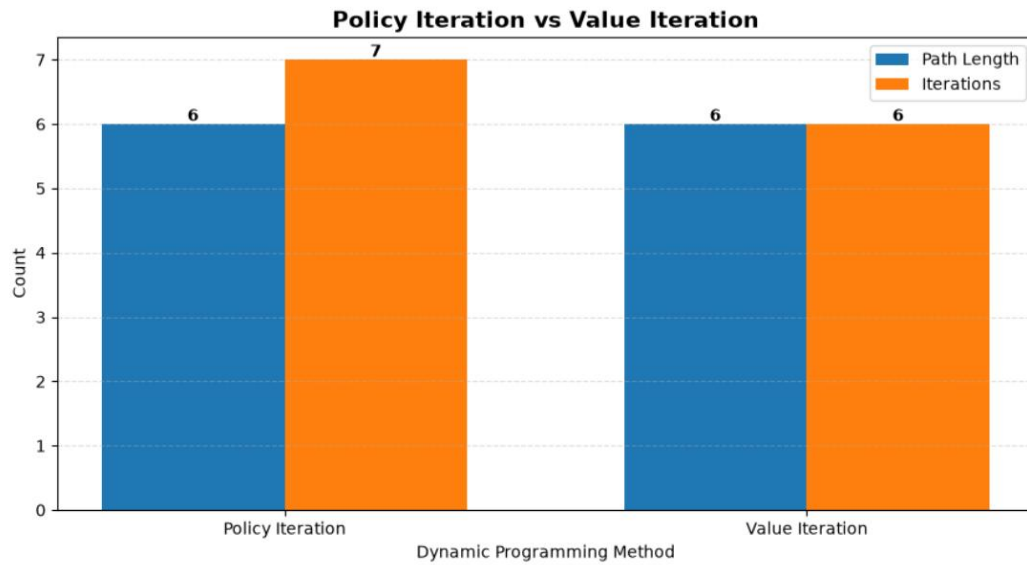
Summary

SUMMARY			
Method	Iterations	Path Length	Start Value
0 Policy Iteration	7	6	-4.095
1 Value Iteration	6	6	-4.095

STATE VALUE COMPARISON			
State	Policy Iteration	Value Iteration	
0 S1	-4.10	-4.10	
1 S2	-3.44	-3.44	
2 S3	-2.71	-2.71	
3 S4	-1.90	-1.90	
4 S5	-3.44	-3.44	
5 S6	-2.71	-2.71	
6 S7	-1.90	-1.90	
7 S8	-1.00	-1.00	
8 S9	-2.71	-2.71	
9 S10	-1.90	-1.90	
10 S11	-1.00	-1.00	
11 S12	0.00	0.00	
12 S13	-1.90	-1.90	
13 S14	-1.00	-1.00	
14 S15	0.00	0.00	
15 S16	0.00	0.00	

Visualization:





Result

Policy Iteration and Value Iteration were successfully implemented. Both methods found the path from **START to GOAL**, and their **iterations and path lengths were compared** using a summary table, Grid World visualization, and comparison graph.